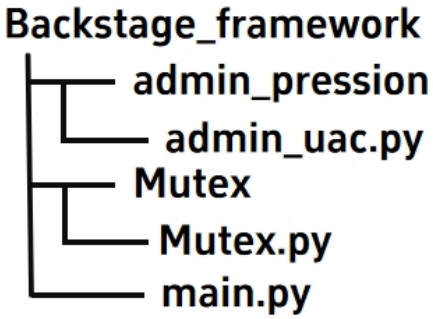
Python-mutex project

嗨，你们好呀，我是小智，欢迎使用 python-mutex 项目，本次基于 windows api 实现互斥锁代码，接下来让我们开始介绍吧 ~

当前版本：v1.0

项目结构如下(需要将 Backstage_framework 文件夹放在与你调用脚本的同目录跟下)：



其中，admin_pression.py 适用于检查管理员/非管理员权限的！Mutex.py 是互斥锁的核心，main.py 负责调用

完整的代码可看 admin_uac.py 源码，讲解如下：

```python
# main.py
from Backstage_framework.admin_pression import admin_uac

check_admin = admin_uac.Check_the_administrator()
if check_admin.process_admin(): # local_admin()
    main()
else:
    check_admin.relaunch_admin()
```

在函数的调用中，check_admin.process_admin 用于检查当前进程是否已经提权，如果是运行，如果没有，就使用 relaunch_admin()重启自己然后拉起 uac 提权，最后管理员身份运行.....

你也可以是用 local_admin()来检查,可以将 process_admin()改为 local_admin()函数，但是风险你需要知道：

```
# 如果当前登陆的 Windows 账户不在管理员组中(SID)，那么 uac 可能会失效，需要输入密码进行确认，这个函数可行，但是条件：当前登录的账户需要在管理员组中，正常情况推荐使用：process_admin()函数，检查当前进程是否已提取！
# 默认你需要管理员权限运行，因为默认是 local\\会话，非管理员需要加 Global\\
```

互斥锁：

```python
# main.py

from Backstage_framework.Mutex import Mutex
from Backstage_framework.admin_pression import admin_uac
import sys

def main():
    input('已经运行')

if __name__ == '__main__':
    app = Mutex.Ultimate_mutex()
    check_admin = admin_uac.Check_the_administrator()

    mutex_handle,exits = app.check_the_ultimate_mutex_lock('我的互斥名字') # 如果是非管理员运行，需要加上 Global\\
    if exits:
        sys.exit(1)

    if check_admin.process_admin(): # 当前是否提权
        if app.ultimate_defense_against_process_hooks():
            try:
                main() # 你的 main()
            finally:
                app.release_mutex()
                app.close_mutex()
    else:
        check_admin.relaunch_admin() # 没有就重启自己管理员运行
```

当你调用一个 check_the_ultimate_mutex_lock()时候，命名空间也随时待命了，括号里需要传入字符串当作你的互斥名字，如果你需要在非管理员下运行，那么你必须加上 Global\\，因为 Windows 默认是 Loacl\\会话，否则会静默失败

同时，当你在调用一个为 ultimate_defense_against_process_hooks 钩子的时候，那么会同步这个锁，进程存在，那么这个锁就存在，但是需要释放锁+管理句柄，因此这里需要放进 finally 函数，确保无论成功/失败，那么这个锁都能正常的清理掉，谢谢！

```python
app = Mutex.Ultimate_mutex()

mutex_handle,exits = app.check_the_ultimate_mutex_lock('我的互斥名字')
if exits:
    sys.exit(1)
```

这个地方一定要放在你的 main()之前，这样才能起到在 UAC 创建之前拦截多开！

实际使用，比如：

```
from Backstage_framework.Mutex import Mutex
from Backstage_framework.admin_pression import admin_uac
import sys

import tkinter as tk

def main():
   root = tk.Tk()
   root.mainloop()

if __name__ == '__main__':
    app = Mutex.Ultimate_mutex()
    check_admin = admin_uac.Check_the_administrator()

    mutex_handle,exits = app.check_the_ultimate_mutex_lock('Global\\我的互斥名字') # 如果是非管理员运行，需要加上 Global\\
    if exits:
        sys.exit(1)

    if check_admin.process_admin():
        if app.ultimate_defense_against_process_hooks():
            try:
                main()
            finally:
                app.release_mutex()
                app.close_mutex()
    else:
        check_admin.relaunch_admin()

这样互斥锁也会生效的，它会在第二个进程或 UAC 创建之前拦截做进一步处理！
```